

PYTHON

Module 17: Dictionaries & Data Mapping

CODING

Introduction to Dictionaries

Dictionaries are Python's implementation of an associative array or map. They store data in 'key: value' pairs, making them highly efficient for looking up information based on a specific identifier.

Core Properties at CodeHive:

- ****Ordered****: As of Python 3.7, dictionaries maintain the order of items based on when they were inserted.
- ****Changeable****: You can add, remove, or update items after the dictionary is created.
- ****Unique Keys****: Dictionaries cannot have duplicate keys.

Assigning a value to an existing key will overwrite the old value.

General Syntax: `thisdict = { "key": "value" }`

Accessing Dictionary Data

Retrieving values from a dictionary can be done in two main ways:

- ****Square Brackets**:**

`dict["key"]` . This will raise an error if the key doesn't exist.

- ****The `.get()` Method**:** `dict.get("key")` . This is safer as it returns `None` instead of an error if the key is missing.

****Viewing the Structure**:**

- `keys()` : Returns a list-like view of all keys.

- `values()` : Returns a list-like view of all values.

- `items()` : Returns a view of key-value pairs as tuples inside a list.

Modifying & Adding Items

Dictionaries are dynamic and allow for easy structural changes.

• ****Direct Entry****: `thisdict["new_key"] = "new_value"` adds a new item or updates an existing one.

• ****The .update() Method****: Merges another dictionary or an iterable of pairs into the current one. If the key exists, it updates; if not, it adds.

Tip: Use `.update()` when you need to change multiple values in a single high-performance operation.

Removal Procedures

There are several ways to delete data depending on your goal:

- `pop("key")` : Removes the specified key and returns its value.

- `popitem()` : Removes the last inserted item in the dictionary.

- `del dict["key"]` : Deletes a specific item permanently.

- `clear()` : Wipes every item, leaving the dictionary empty (`{}`).

- `del dict` : Deletes the entire dictionary variable from system memory.

Advanced Iteration

Looping through dictionaries offers different levels of access:

- **Looping Keys**: `for x in thisdict:`

- **Looping Values**: `for x in thisdict.values():`

- **Looping Both**: `for key, value in thisdict.items():`

At CodeHive, we recommend using `.items()` for the most readable code when you need both the label and the data.

Nested Dictionaries

Dictionaries can contain other dictionaries, allowing for complex, hierarchical data structures similar to JSON.

Example Structure:

python

```
my_api_response = {  
    "user": { "id": 1, "name": "John" },  
    "status": "success"  
}
```

To access nested data, chain the keys:
`my_api_response["user"]["name"] .`

Copying & Data Integrity

Warning: `dict2 = dict1` creates a reference, not a copy. Changing one will change the other.

CodeHive Safety Methods:

- `new_dict = original.copy()`

- `new_dict = dict(original)`

Always use these methods when you want to manipulate data without affecting the source dictionary.

Dictionary Method Reference

Essential tools for the professional:

-

`fromkeys()` : Creates a dictionary with specified keys and a unified value.

-

`setdefault()` : Returns value of a key, but inserts it with a default if it's missing.

Dictionaries are the backbone of data exchange in Python, especially when dealing with web APIs and database records.